TU Wien
Institut für Logic and Computation
Forschungsbereich Algorithms and Complexity

**186.866 Algorithmen und Datenstrukturen VU**

# Programmieraufgabe P3

PDF erstellt am: 24. März 2026

## 1 Vorbereitung

Um diese Programmieraufgabe erfolgreich durchführen zu können, müssen folgende Schritte umgesetzt werden:

1. Laden Sie das zugehörige Java-Projekt `P3.zip` aus TUWEL runter.
2. Entpacken Sie `P3.zip` und öffnen Sie das entstehende Projektverzeichnis, d.h. das *äußere* Verzeichnis `task`, in Ihrer Entwicklungsumgebung.
3. Öffnen Sie die folgende Datei in Ihrem Editor: `task/src/main/java/StudentSolutionForExerciseImplementation.java`
   In dieser Datei sind sämtliche Programmiertätigkeiten durchzuführen. Ändern Sie keine anderen Dateien im Projekt und fügen Sie auch keine neuen hinzu.
4. Füllen Sie Vorname, Nachname und Matrikelnummer in der Methode `StudentInformation provideStudentInformation()` aus.
5. Gehen Sie sicher, dass Java-Version 21 auf Ihrem System verfügbar ist.

   Alternativ zu einer manuellen Installation, können Sie in der Datei `settings.gradle` das auskommentierte Code-Snippet für eine automatische Installation nutzen.
6. Details zur Ausführung des Codes finden Sie in Abschnitt 7.

## 2 Hinweise

Einige Hinweise, die Sie während der Umsetzung dieser Aufgabe beachten müssen:

- Lösen Sie die Aufgaben selbst. Verwenden Sie auch keine (KI-)Tools z. B. zur Codegenerierung.
- Sie dürfen beliebig viele Hilfsmethoden schreiben und benutzen. Beachten Sie aber, dass Sie nur die oben geöffnete Datei abgeben und diese Datei mit dem zur Verfügung gestellten Framework lauffähig sein muss.
- Es ist grundsätzlich erlaubt, Bibliotheken wie z. B. das Java Collections Framework und die Java Math Klasse zu verwenden. Die Voraussetzung dabei ist, dass die Verwendung die eigentliche Programmieraufgabe nicht abnimmt. Beachten Sie, dass nur die oben genannte Java-Version auf unserem Testsystem zur Verfügung steht.

# 3 Übersicht

In dieser Programmieraufgabe beschäftigen wir uns mit dem *KD-Tree* (kurz für *k-dimensional tree*), einer hierarchischen Datenstruktur zur effizienten Speicherung und Abfrage von mehrdimensionalen Punkten. Der Fokus liegt dabei auf dem zweidimensionalen Fall, wie er häufig in Anwendungen der Computational Geometry, der Geoinformatik oder der Grafikprogrammierung vorkommt.

KD-Trees ermöglichen insbesondere eine schnelle Durchführung von Bereichsanfragen sowie Nachbarschaftssuchen und stellen damit eine wichtige Alternative zu linearen Suchverfahren dar.

# 4 Theorie

Die für diese Aufgabe notwendige theoretische Grundlage, für einfache Suchbäume, kann den Vorlesungsfolien „Suchbäume“ entnommen werden. In diesem Abschnitt fassen wir die zentralen Begriffe und Definitionen zusammen, die für das Verständnis und die Implementierung von KD-Trees erforderlich sind.

Ein KD-Tree ist eine binäre Baumstruktur, in der jeder innere Knoten den Raum anhand einer ausgewählten Koordinate in zwei Teilräume unterteilt. Im zweidimensionalen Fall wechseln sich die Aufteilungen entlang der $x$- und $y$-Achse ab. Jeder Knoten speichert also eine Trennlinie, die den Raum in

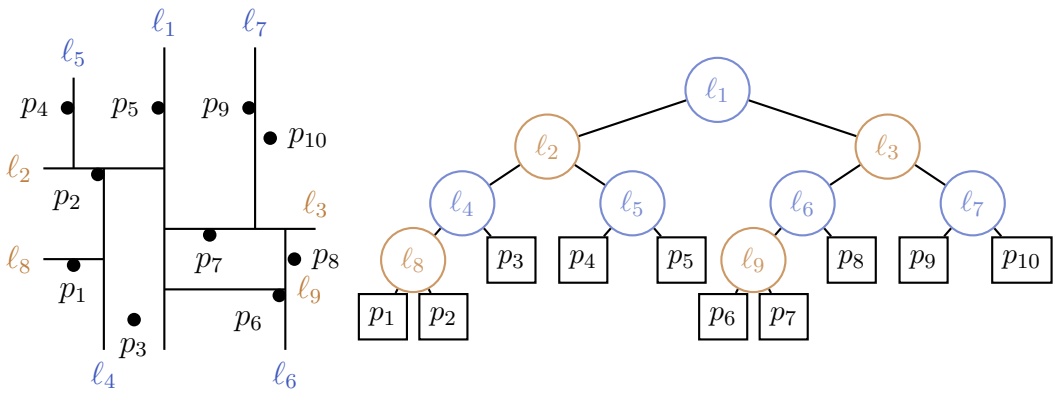


Abbildung 1: Beispiel einer Punktanordnung (links) und des zugehörigen *KD-Trees* (rechts).

einen linken und einen rechten oder einen oberen und unteren Teilbereich zerlegt.

Die Richtung der Trennlinie hängt von der Tiefe des Knotens im Baum ab:

- **Gerade Baumtiefen** (z. B. 0, 2, 4, ...) entsprechen **vertikalen Trennlinien**, die entlang der $x$-Achse teilen. Liegt ein Punkt links oder auf der vertikalen Linie, so wird er im linken Teilbaum gespeichert, liegt er rechts, so wird er im rechten Teilbaum gespeichert.
- **Ungerade Baumtiefen** (z. B. 1, 3, 5, ...) entsprechen **horizontalen Trennlinien**, die entlang der $y$-Achse teilen. Liegt ein Punkt unterhalb oder auf der horizontalen Linie, so wird er im linken Teilbaum gespeichert, liegt er oberhalb, so wird er im rechten Teilbaum gespeichert.

Die Wurzel des Baumes befindet sich auf Tiefe 0 und erzeugt somit eine vertikale Trennlinie. Durch diese alternierende Aufteilung entsteht eine rekursive Zerlegung des zweidimensionalen Raumes, die effiziente Such- und Bereichsanfragen ermöglicht. Die Blattknoten des Baumes entsprechen den finalen Punkten, die gespeichert werden.

## 4.1 Beispiel:

In Abbildung 1 ist ein Beispiel einer Punktwolke und des entsprechenden *KD-Trees* gegeben. Vertikale Linien liegen auf geraden Baumtiefen, horizontale

Linien auf ungeraden Baumtiefen. Jeder Punkt entspricht einem Blatt im Baum und ihm ist sein eigener Bereich im Raum zugeordnet.

# 5 Operationen im KD-Tree

## 5.1 Einfügen eines Punktes

Das Einfügen eines Punktes in einen KD-Tree erfolgt rekursiv, beginnend an der Wurzel:

- Falls der Teilbaum leer ist, wird an dieser Stelle ein neuer Blattknoten mit dem Punkt angelegt.
- Falls der Teilbaum aus einem Blattknoten besteht, wird eine entsprechende Trennlinie eingefügt. Diese verläuft entlang des Mittelwerts der Koordinatenwerte der beiden aktuell betrachteten Punkte. Abhängig von der Tiefe des Baums wird dabei die $x$- oder die $y$-Koordinate verwendet.

  Der Spezialfall, dass beide Knoten denselben Koordinatenwert besitzen und somit im nächsten Schritt weiter unterteilt werden müssten, tritt in unserer Anwendung nicht auf.
- Handelt es sich um einen inneren Knoten, wird die Koordinate des einzufügenden Punktes mit der Trennlinie des aktuellen Knotens verglichen:
  - Bei einer **vertikalen Linie** (gerade Tiefe) wird die $x$-Koordinate verglichen.
  - Bei einer **horizontalen Linie** (ungerade Tiefe) wird die $y$-Koordinate verglichen.

  Liegt der Punkt links von oder auf der Trennlinie (bzw. unterhalb oder auf der Linie bei einer horizontalen Trennung), wird der linke Teilbaum weiterverfolgt. Andernfalls wird in den rechten Teilbaum gewechselt.

Durch diese rekursive Einfügemethode bleibt die Struktur des KD-Trees konsistent und die alternierende Trennung entlang der Achsen wird beibehalten.

## 5.2 Bereichsanfrage

Eine Bereichsanfrage (engl. *Range Query*) sucht alle Punkte aus einem Baum, die innerhalb eines gegebenen Rechtecks liegen. Der Algorithmus arbeitet ebenfalls rekursiv und wird stets auf einem Teilbaum des *KD-Trees* aufgerufen:

- Ist die Wurzel des aktuellen Teilbaums ein Blattknoten, wird überprüft, ob der darin gespeicherte Punkt im gesuchten Bereich liegt.
- Handelt es sich um einen inneren Knoten, muss entschieden werden, ob nur der linke, der rechte oder beide Teilbäume durchsucht werden.

  Für die folgende Beschreibung wird von einer vertikalen Trennlinie ausgegangen; der Algorithmus funktioniert jedoch analog für horizontale Linien.

  - Falls die linke Seite des Suchrechtecks die Trennlinie berührt oder links von ihr liegt, wird der linke Teilbaum durchsucht.
  - Falls die rechte Seite des Suchrechtecks rechts von der Trennlinie liegt, wird der rechte Teilbaum durchsucht.

Durch diese selektive Rekursion werden nur diejenigen Teilbäume untersucht, die potenziell Punkte im Suchbereich enthalten, was die Effizienz der Suche erhöht. Ist der Baum leer oder das Rechteck so klein, dass keine Punkte darin vorkommen, soll eine leere Liste zurückgegeben werden.

# 6 Implementierung

Sie haben Zugang zu den Datenstrukturen `Point`, dem abstrakten Datentyp `KDNode` sowie den konkreten Subtypen `KDLeafNode`, `KDInnerNode` und dem Gesamtdatentyp `KDTree`. Nachfolgend wird eine Beschreibung der Methoden gegeben, die auf diesen Datenstrukturen zur Verfügung stehen.

Die Klasse `Point` verwaltet einen Punkt sowie dessen $x$- und $y$-Koordinaten und bietet folgende Methoden an:

- `double getX()`: Gibt die $x$-Koordinate zurück.
- `double getY()`: Gibt die $y$-Koordinate zurück.

Die Klasse `KDLeafNode` verwaltet die Blattknoten des Baumes. Ein Blattknoten kann wie folgt instanziiert werden:

- `KDLeafNode(Point point)`

und bietet folgende Methode an:

- `Point getPoint()`: Gibt den im Blattknoten gespeicherten Punkt zurück.

Die Klasse `KDInnerNode` verwaltet die inneren Knoten des *KD-Trees*. Ein innerer Knoten kann wie folgt instanziiert werden:

- `KDInnerNode(int depth, double coordinate, KDNode left, KDNode right)`

und bietet folgende Methoden an:

- `int getDepth()`: Gibt die Tiefe des Knotens im Baum zurück.
- `double getCoordinate()`: Gibt den Koordinatenwert zurück, entlang dessen der Raum geteilt wird.
- `KDNode getLeft()`: Gibt die Wurzel des linken Teilbaums zurück.
- `KDNode getRight()`: Gibt die Wurzel des rechten Teilbaums zurück.
- `void setLeft(KDNode node)`: Setzt den linken Teilbaum auf den gegebenen Knoten.
- `void setRight(KDNode node)`: Setzt den rechten Teilbaum auf den gegebenen Knoten.

Die Klasse `KDTree` repräsentiert den gesamten *KD-Tree*. Diese Datenstruktur verwaltet sowohl die Wurzel des Baumes als auch die Gesamtheit aller Punkte und bietet folgende Methoden an:

- `void setRoot(KDNode node)`: Setzt die Wurzel des Baumes auf den gegebenen Knoten.
- `KDNode getRoot()`: Gibt die Wurzel des Baumes zurück.

## 6.1 Einfügen von Punkten

Implementieren Sie zunächst die Methode `void insert(Point point, KDTree tree)`. Die Methode soll die beschriebene Einfügestrategie in Abschnitt 5.1 implementieren.

## 6.2 Bereichsanfrage

Entwickeln Sie eine Methode `Point[] computeRangeQuery(KDTree tree, double left, double right, double top, double bottom)`, die die Bereichsanfrage wie in Abschnitt 5.2 beschrieben berechnet.

Implementieren Sie außerdem eine Methode `Point[] computeRangeQueryNaive(Point[] pointArray, double left, double right, double top, double bottom)`, die naiv alle Punkte iterativ überprüft und jene zurückgibt, die in den gefragten Bereich fallen.

Sie können davon ausgehen, dass `left < right` und `bottom < top`.

# 7 Testen

Führen Sie im Projektverzeichnis den Befehl `./gradlew run` aus (`.\gradlew.bat run` unter Windows).

Sollten Sie IntelliJ nutzen, können Sie das Projekt auch direkt über die grafische Oberfläche starten. Klicken Sie dazu in der rechten Leiste auf das Gradle-Logo (Elefant) und navigieren Sie zum „run“-Task (siehe Abbildung 2). Mittels Rechtsklick bzw. Sekundärklick können Sie das Projekt ausführen („Run“) oder auch den visuellen Debugger nutzen („Debug“). Die Option „Modify Run Configuration...“ erlaubt es Ihnen, die Ausführung permanent als „Run/Debug Configuration“ zu hinterlegen.

Anschließend wird Ihnen in der Konsole eine Auswahl an Testinstanzen angeboten, darunter befindet sich zumindest `abgabe.csv`:

```
Select an instance set or exit:
[1] abgabe.csv
[0] Exit
```

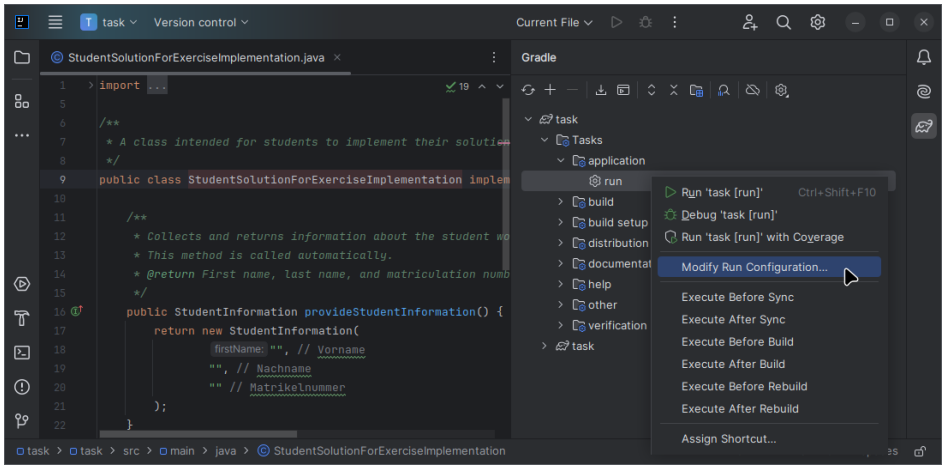


Abbildung 2: Ausführen von Gradle-Tasks in IntelliJ

Durch die Eingabe der entsprechenden Ziffer kann entweder eine Testinstanz ausgewählt werden oder das Programm (mittels der Eingabe von `0`) verlassen werden. Wird eine Testinstanz gewählt, dann wird der von Ihnen implementierte Programmcode ausgeführt. Kommt es dabei zu einem Fehler, wird ein Hinweis in der Konsole ausgegeben.

Um nur eine bestimmte Subinstanz auszuführen, hängen Sie `:[Subinstanznummer]` an Ihre Auswahl an (z. B. `1:123`).

Relevant für die Abgabe ist das Ausführen der Testinstanz `abgabe.csv`.

# 8 Evaluierung

Wenn der von Ihnen implementierte Programmcode mit der Testinstanz `abgabe.csv` ohne Fehler ausgeführt werden kann, dann wird nach dem Beenden des Programms im Verzeichnis `results` eine Ergebnisdatei mit dem Namen `solution-abgabe.csv` erzeugt. Beachten Sie dabei, dass Sie die gesamte Testinstanz ausführen und nicht nur eine Subinstanz.

Die Datei `solution-abgabe.csv` beinhaltet Zeitmessungen der Ausführung der Testinstanz `abgabe.csv`, welche in einem Webbrowser visualisiert werden können. Auch Ergebnisdateien anderer Testinstanzen können zu Testzwecken visualisiert werden. Öffnen Sie dazu die Datei `visualization.html` in Ihrem Webbrowser und klicken Sie rechts oben auf den Dateiauswahlknopf, um `solution-abgabe.csv` auszuwählen.

Beantworten Sie basierend auf der Visualisierung die Fragestellungen aus dem folgenden Abschnitt.

# 9 Fragestellungen

Öffnen Sie `solution-abgabe.csv` und bearbeiten Sie folgende Aufgaben- und Fragestellungen:

1. Betrachten Sie den Laufzeitplot. Auf der x-Achse ist die Anzahl der Punkte und auf der y-Achse die Zeit für die Abfrage dargestellt.

   Vergleichen Sie die Laufzeiten von `computeRangeQuery` und `computeRangeQueryNaive`. Um die Ergebnisse stabiler zu machen, werden die Experimente mehrfach wiederholt; die Zahl am Ende jeder Gruppe gibt an, wie oft das jeweilige Experiment durchgeführt wurde.

   Beantworten Sie insbesondere folgende Fragestellungen:

   - Welche Auswirkungen hat die Anzahl der Punkte auf die Laufzeit?
   - Welches asymptotische Laufzeitverhalten beobachten Sie?
   - Welches Laufzeitverhalten hätten Sie erwartet?

   Speichern Sie im Anschluss den Plot als Bild, indem Sie in der Menüleiste rechts über dem Plot auf den Fotoapparat drücken.

2. Erklären Sie, wie man einen *KD-Tree* auf mehr als 2-Dimensionen erweitern kann.

3. Lassen Sie sich alle Punkte der ersten Instanz im Terminal/Debugger ausgeben. Wie wird der Baum, der aus diesen Punkten entsteht, aussehen?

4. Nehmen Sie an, dass alle Punkte im Vorhinein bekannt sind. Wie kann schon beim Erstellen der Baumes das Problem, das beim Baum aus Punkt 3 auftritt, vermieden werden?

Falls sich im Zuge der Evaluierung die Darstellung der Plots auf ungewünschte Weise verändert (z. B. durch die Auswahl eines zu kleinen Ausschnitts), können Sie mittels Doppelklick auf den Plot oder Klick auf das Haus in der Menüleiste die Darstellung zurücksetzen.

Fügen Sie Ihre Antworten in einem Bericht gemeinsam mit allen erstellten Bildern der Visualisierungen der Testinstanz `abgabe.csv` zusammen.

# 10 Abgabe

Laden Sie die Datei `task/src/main/java/StudentSolutionForExerciseImplementation.java` in der TUWEL-Aktivität *Hochladen Source-Code P3* hoch. Fassen Sie diesen Bericht mit den anderen für das zugehörige Abgabegespräch relevanten Berichten in einem PDF zusammen und geben Sie dieses in der TUWEL-Aktivität *Hochladen Bericht Abgabegespräch 1* ab.

# 11 Nachwort

Die *KD-Tree*-Datenstruktur ist ein fundamentales Werkzeug in der Computational Geometry und findet breite Anwendung in Bereichen wie der Geoinformatik, der Grafikprogrammierung und der Datenanalyse. Durch die effiziente Organisation von mehrdimensionalen Punkten ermöglicht sie schnelle Such- und Abfrageoperationen, die in vielen praktischen Anwendungen von entscheidender Bedeutung sind.

Auch in der künstlichen Intelligenz, insbesondere im maschinellen Lernen, werden KD-Trees häufig verwendet, um schnelle Nachbarschaftssuchen durchzuführen, beispielsweise bei der Klassifikation oder Regression mittels k-nearest neighbors Algorithmen. In *Retrieval Augmented Generation* (RAG) Systemen, die oft an Large Language Modelle gekoppelt werden, tragen KD-Trees als fundamentale Datenstruktur in Vektordatenbanken dazu bei, relevante Informationen aus großen Datenmengen effizient abzurufen und zu verwenden.

Weiterführendes hierzu können Sie in unseren folgenden LVAs erfahren:

186.814 VU Algorithmics
192.154 VU Algorithmic Geometry